

COMP.SE.140-2024-2025-1 Continuous Development and Deployment -
DevOps

Nginx hands-on

Report

Eero Santala 151179055

30.1.2025

Table of Contents

Assumptions of the exercise.....	2
Description of the system.....	2
Instructions for teaching assistant.....	3
Implemented optional features.....	3
Instructions for examiner to test the system.	3
Computer specs used in developing and running the system.....	3
How to run the system	3
Description of the CI/CD pipeline.....	4
Version management.....	4
Building tools.....	5
Testing tools	5
Packing	5
Deployment.....	6
Operating; monitoring	6
Example runs of the pipeline	7
Below is an image of failing tests.....	7
And then passing tests	8
Test Driven Development in the pipeline.....	9
Deployment in the pipeline	9
Broken pipeline	10
Reflections	10
Ammount of effort	11

Assumptions of the exercise

The requirements were a little ambiguous on some parts of the system by design, so here are the made assumptions on how I interpreted some of them and reasoning for design choices I made.

PUT /state

- Changing to Init is allowed from other states. Doesn't logout browser, as browser will just log back in.
- From init shutdown or paused is not allowed. Running is allowed to have it all in one functionality. Login calls PUT running instead of another endpoint.
- From running all changes are allowed
- Only login changes Init to "Running". PUT /state is not allowed to change Init → Running

GET /state

- responds always

GET /request

- If system state is not "Running" or "Init" → empty response with jovial 418 code.
- Else responds with status code 200 with service1 and service2 responses.

GET /run-log

- responds always

Description of the system

I went a bit overboard in designing the system. I ended up adding "gateway" service to handle api endpoints and "stateservice" to handle database access and data persistence. The goal was to add database to a persistent volume in csc virtual machine, but it was unfinished. In hindsight this only added to complexity and once I started to have difficulties with the CI/CD pipeline I was kicking myself for making it unnecessary complex. This kind of system could have been better overall if we were allowed to change nginx basic authentication to a token based authentication as it would have made logging out from browser with api much easier. System is illustrated in Figure 1.

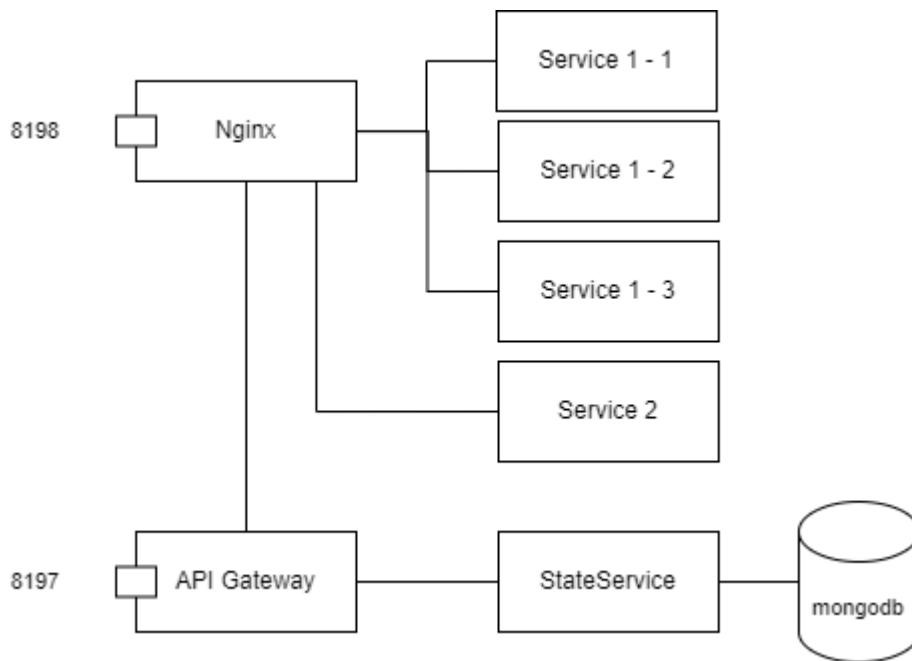


Figure 1. Deployment diagram.

Instructions for teaching assistant

Implemented optional features

Tests cover some individual services too.

~~Deployment to csc cloud platform virtual machine~~

Instructions for examiner to test the system.

Computer specs used in developing and running the system

CPU: Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz

RAM: 16 Gt

Operation systems used:

Windows 10 Home 22H2 19045.5371

Docker version 25.0.3, build 4debf41

Docker Compose version v2.24.6-desktop.1

Ubuntu 24.04

How to run the system

```
git clone -b project git@github.com:EeroSan/Nginx_SE140.git
```

After cloning the repository from branch “project” create an environment variable file ‘.env’

with the following contents:

MONGO_USER=placeholderuser

MONGO_PASSWORD=placeholderpassword

MONGO_IP=mongo

MONGO_DB=mongodb

MONGO_PORT=27017

API_KEY=placeholderapikey

MONGO_COLLECTION=placeholdercollection

Then the system can be brought up by

`docker-compose up --build`

Login credentials: username harjoitusharri password Harrinsalasana123

Some curl commands used in testing

Notice that some of the “ are translated incorrectly on ubuntu when copied from here.

`curl -u harjoitusharri:Harrinsalasana123 -v http://localhost:8198/`

`curl localhost:8197/state -X PUT -d "PAUSED" -H "Content-Type: text/plain" -H "Accept: text/plain"`

`curl localhost:8197/state -X PUT -d "INIT" -H "Content-Type: text/plain" -H "Accept: text/plain"`

`curl -u harjoitusharri:Harrinsalasana123 -v http://localhost:8198/`

`curl localhost:8197/request -X GET -H "Content-Type: text/plain" -H "Accept: text/plain"`

`curl localhost:8197/run-log -X GET -H "Content-Type: text/plain" -H "Accept: text/plain"`

`curl localhost:8197/state -X PUT -d "SHUTDOWN" -H "Content-Type: text/plain" -H "Accept: text/plain"`

Description of the CI/CD pipeline

Version management

All of the development were done in “project” branches. Two repositories were used

Github: https://github.com/EeroSan/Nginx_SE140/tree/project

GitLab: https://compse140.devops-gitlab.rd.tuni.fi/santala/devops_project

GIT Bash was primarily used for working with the repositories. GitLab was setup to act as a CI/CD pipeline and it mirrored Github repository when a push was made.

Building tools

Node 21, npm 10.5.0, ESLint, Docker, Docker Compose, GitLab CI/CD, Ubuntu Linux, Windows

Gitlab runner was installed on a local virtual machine running Ubuntu server 22. On hindsight it would have been better to deploy it in cdc cloud platforms virtual machine. Then it would have ran even when the local machine is not up and running.

Testing tools

For nodejs based services (service1, gateway and stateservice) testing the following tools were used

- mocha 10.0.0
- chai 4.3.7
- supertest 6.3.3
- nock 13.0.0
- sinon 15.0.0

The responses from other services were mocked using nock and supertest, this was to reduce headache of running the tests in the CI/CD pipeline.

For testing python (service2) pytest were used

Test cases

Testing was done for each service because it was advised to be kept barebones, thus integration testing between services was not done. Integration testing in the pipeline seemed like a nightmare that I was relieved to skip. Test-Driven-Development was required, but there were not many modifications to service1 and service2 thus resulting poor tests for them.

StateService tests focus on verifying login state management and system state updates. The tests check if login status can be correctly retrieved and updated, returning appropriate responses when the state is uninitialized. Similarly, the system state tests confirm that valid updates are accepted while invalid inputs are rejected, ensuring robust error handling. These tests rely on Sinon for stubbing database operations and Nock to mock MongoDB interactions, preventing dependencies on an actual database.

For API gateway tests validate that the gateway correctly fetches and updates system states, logs system activity, and processes requests from various services. By using Nock to mock service responses and Supertest to make API requests, these tests ensure that the gateway behaves correctly under different conditions.

Packing

I implemented 4 stages: Synchronization => Build => Test => Deploy and planned for Deploying cdc which was not implemented, but it would have been triggered by pushing a version tag into the repository.

On the **synchronization** stage the gitlab repository is mirrored with github repository in order to keep the repository contents same, even when a lazy developer forgets to push into the remote.

Building stage confirms that all services are able to be built.

Testing stage confirms that all the tests are ran successfully and fails the pipeline even if one test case fails. This ensures regression testing.

On **the Deployment** stage services are brought up and deployed locally on a virtual machine where gitlab runner is located.

Deployment

Deployment was done locally on the virtual machine with the simplest setup with docker-compose.

Operating; monitoring

This optional feature was not implemented.

Example runs of the pipeline

Below is an image of failing tests



🔍

🔖

📄

Q Search or go to...

Project

devops_project

Pinned

Issues

Merge requests

Manage

Plan

Code

Build

Pipelines

Jobs

Pipeline editor

Pipeline schedules

Artifacts

Secure

Deploy

Operate

Monitor

Analyze

Settings

Help

test

Failed Started 1 week ago by Eero Santala

Search visible log output

```
1 Running with gitlab-runner 17.7.0 (3151cccd)
2 on devops_t3_7095zj, system ID: s_9a0d61cac896
3 Preparing the "docker" executor
19 Preparing environment
21 Getting source from Git repository
27 Downloading artifacts
28 Downloading artifacts for build (8734)...
29 Downloading artifacts from coordinator... ok
30 host=compse1408.devops-gitlab.rdt.tuni.fi id=8736 responseStatus=200 OK tokens=glcbrt-64
31 Executing "stop_script" stage of the job script
32 Using docker image sha256:b300b8dc5f2ac4f188ecdd418f3ea30e35b4ee1018ff818ca39dd3f0bc7aadd for docker:latest with digest docker@sha256:45b88efc6e892498e116
33 New "kubernetes" file structure:
34 Checking file structure:
35 $ export CI_PROJECT_DIR=/PMD
36 $ echo "kurva CI_PROJECT_DIR is set to $(CI_PROJECT_DIR)"
37 kurva CI_PROJECT_DIR is set to /builds/santala/devops_project
38 $ ls -l $(CI_PROJECT_DIR)/tests/service2_tests
39 total 8
39 drwxr-xr-x 2 root root 4096 Jan 21 15:22 __pycache__
40 -rw-r--r-- 1 root root 548 Jan 21 15:22 test_app.py
41 $ echo "Running tests..."
42 Running tests...
43 $ echo "Running tests for Service 1..."
44 Running tests for Service 1...
45 $ docker run --rm -v /PMD/tests/service1_tests:/app/tests test_service1 || exit 1
46 > Service 1@1.0.0 test
47 > mocha test --reporter mocha-junit-reporter --reporter-options mochaFile=./test-reports/test-results.xml
48 $ echo "Running tests for Service 2..."
49 Running tests for Service 2...
50 $ docker run --rm -v /PMD/tests/service2_tests:/app/tests test_service2 || exit 1
51 ===== test session starts =====
52 platform linux -- Python 3.9.21, pytest-8.3.4, pluggy-1.5.0
53 rootdir: /app
54 plugins: flask-1.3.0
55 collected 1 item
56 Service2/test_app.py . [100%]
57 ===== 1 passed in 0.14s =====
58 $ echo "Running tests for Gateway..."
59 Running tests for Gateway...
60 $ docker run --rm -v /PMD/tests/gateway_tests:/app/tests test_gateway || exit 1
61 > Gateway@0.0.0 test
62 > mocha test
63 API Gateway Tests
64 gatewayPath /src/gateway
65 Skipping app.listen
66 GET /state
67 Received state request
68 Attempting to get state
69 ✓ should return the current state
70 PUT /state
71 1) should update the state to INIT
72 2) should update the state to PAUSED
73 3) should update the state to RUNNING
74 4) should update the state to SHUTDOWN
75 1 passing (9ms)
76 4 failing
77 1) API Gateway Tests
78 PUT /state
79 should update the state to INIT:
80 AssertionError: expected 404 to equal 200
81 + expected - actual
82 -404
83 +200
84
85 at Context.<anonymous> (/test/gateway_test.js:27:31)
86 at process.processTicksAndRejections (node:internal/process/task_queues:95:5)
87 2) API Gateway Tests
88 PUT /state
89 should update the state to PAUSED:
90 AssertionError: expected 404 to equal 200
91 + expected - actual
92 -404
93 +200
94
95 at Context.<anonymous> (/test/gateway_test.js:33:31)
96 at process.processTicksAndRejections (node:internal/process/task_queues:95:5)
97 3) API Gateway Tests
98 PUT /state
99 should update the state to RUNNING:
100 AssertionError: expected 404 to equal 200
101 + expected - actual
102 -404
103 +200
104
105 at Context.<anonymous> (/test/gateway_test.js:39:31)
106 at process.processTicksAndRejections (node:internal/process/task_queues:95:5)
107 4) API Gateway Tests
108 PUT /state
109 should update the state to SHUTDOWN:
110 AssertionError: expected 404 to equal 200
111 + expected - actual
112 -404
113 +200
114
115 at Context.<anonymous> (/test/gateway_test.js:45:31)
116 at process.processTicksAndRejections (node:internal/process/task_queues:95:5)
117 Cleaning up project directory and file based variables
118 ERROR: Job failed: exit code 1
```

Duration: 13 seconds

Finished: 1 week ago

Queued: 3 seconds

Timeout: 1h (from project)

Trigger token: b328

Trigger variables

Reveal values

Key	Value
TRIGGER_PAYLOAD	AD

Commit bff1b98

Sync from GitHub

Pipeline #2696 Failed for project

test

Related jobs

→ test

And then passing tests



Q Search or go to...

Project

devops_project

Pinned

Issues

Merge requests

Manage

Plan

Code

Build

Pipelines

Jobs

Pipeline editor

Pipeline schedules

Artifacts

Secure

Deploy

Operate

Monitor

Analyze

Settings

Help

Eero Santala / devops_project / Jobs / #8740

test

Passed Started 1 week ago by Eero Santala

Search visible log output

Duration: 42 seconds
Finished: 1 week ago
Queued: 2 seconds
Timeout: 1h (from project)

Trigger token: b328
Trigger variables
Key Value
TRIGGER_PAYLOAD AD *****

Commit 35a46e58
Sync from GitHub

Pipeline #2697 Passed for project
test

Related jobs
→ test

```
1 Running with gitlab-runner 17.7.0 (3153ccc0)
2 on devops t3_7C9SzJ, system ID: s_9a8061cac896
3 Preparing the "docker" executor
4 Using docker executor with image docker:latest ...
5 Starting service docker:19.03.12-dind...
6 Pulling docker image docker:19.03.12-dind ...
7 Using docker image sha256:6dcd2645749a48592f4348fb3d567bdc65c9bd5402a413bd169619a32f6bd2 for docker:19.03.12-dind with digest docker@sha256:674f1f40ff7c8
ac14f5d8bb28d8f1f182647ff75384d818003f1e21a8d8771 ...
8 Waiting for services to be up and running (timeout 30 seconds)...
9 == WARNING: Service runner-t37c9szj-project-60-concurrent-0 probably didn't start properly.
10 Health check error:
11 service "runner-t37c9szj-project-60-concurrent-0-6b438a1e3a213cb0-docker-0-wait-for-service" timeout
12 Health check container logs:
13 2025-01-21T15:27:21.652818744Z waiting for TCP connection to 172.17.0.2 on [2375 2376]...
14 2025-01-21T15:27:21.652880628Z dialing 172.17.0.2:2376...
15 2025-01-21T15:27:21.652918783Z dialing 172.17.0.2:2376...
16 2025-01-21T15:27:25.720669258Z dialing 172.17.0.2:2376...
17 2025-01-21T15:27:25.720689437Z dialing 172.17.0.2:2376...
18 2025-01-21T15:27:29.888793454Z dialing 172.17.0.2:2376...
19 2025-01-21T15:27:29.888813849Z dialing 172.17.0.2:2376...
20 2025-01-21T15:27:33.98414946Z dialing 172.17.0.2:2376...
21 2025-01-21T15:27:33.984142458Z dialing 172.17.0.2:2376...
22 2025-01-21T15:27:37.999097163Z dialing 172.17.0.2:2376...
23 2025-01-21T15:27:37.999118088Z dialing 172.17.0.2:2376...
24 2025-01-21T15:27:42.895586254Z dialing 172.17.0.2:2376...
25 2025-01-21T15:27:42.895687893Z dialing 172.17.0.2:2376...
26 2025-01-21T15:27:46.192497138Z dialing 172.17.0.2:2376...
27 2025-01-21T15:27:46.192517952Z dialing 172.17.0.2:2376...
28 2025-01-21T15:27:50.288553607Z dialing 172.17.0.2:2376...
29 2025-01-21T15:27:50.288575634Z dialing 172.17.0.2:2376...
30 Service container logs:
31 2025-01-21T15:27:21.258373967Z time="2025-01-21T15:27:21.258373967Z" level=info msg="Starting up"
32 2025-01-21T15:27:21.260938818Z time="2025-01-21T15:27:21.260938818Z" level=warning msg="could not change group /var/run/docker.sock to docker: group docker
not found"
33 2025-01-21T15:27:21.264714124Z failed to load listeners: can't create unix socket /var/run/docker.sock: device or resource busy
34 *****
35 Pulling docker image docker:latest ...
36 Using docker image sha256:b108b8cccf2ac4f188eccc418f3ea30e35b4ee1018ff818ca39dd3f0bc7aadd for docker:latest with digest docker@sha256:45b88efce6892498e116
0c3be8eb7cc67732863cac98d2a79af24398ff1c106 ...
37 Preparing environment
38 Running on runner-t37c9szj-project-60-concurrent-0 via cloudftechbuntu...
39 Getting source from Git repository
40 Fetching source code via git depth set to 50
41 Reinitialized existing Git repository in /builds/santala/devops_project/.git/
42 Checking out 35a46e58 as detached HEAD (ref is project)...
43 Removing .docker/
44 Skipping Git submodules setup
45 Downloading artifacts
46 Downloading artifacts for build (8939)
47 Downloading artifacts from coordinator... ok host=compse148.devops-gitlab.rd.tuni.fi id=8739 responseStatus=200 OK token=glcbrt-64
48 Executing "stop_script" stage of the job script
49 Using docker image sha256:b308b8cccf2ac4f188eccc418f3ea30e35b4ee1018ff818ca39dd3f0bc7aadd for docker:latest with digest docker@sha256:45b88efce6892498e116
0c3be8eb7cc67732863cac98d2a79af24398ff1c106 ...
50 $ echo "Checking file structure:"
51 $ export CI_PROJECT_DIR=$PWD
52 $ echo "kurva CI_PROJECT_DIR is set to $(CI_PROJECT_DIR)"
53 $ echo "kurva CI_PROJECT_DIR is set to /builds/santala/devops_project"
54 $ ls -l $(CI_PROJECT_DIR)/tests/service2_tests
55 total 8
56 -rwxr-xr-x 2 root root 4894 Jan 21 15:26 _pycache_
57 -rwxr-xr-x 1 root root 548 Jan 21 15:26 test_app.py
58 $ echo "Running tests..."
59 Running tests...
60 $ echo "Running tests for Service 1..."
61 Running tests for Service 1...
62 $ docker run --rm -v $PWD/tests/service2_tests:/app tests test_service1 || exit 1
63 > Service 1@1.0.0 test
64 > mocha test --reporter mocha-junit-reporter --reporter-options mochaFile=./test-reports/test-results.xml
65 $ echo "Running tests for Service 2..."
66 Running tests for Service 2...
67 $ docker run --rm -v $PWD/tests/service2_tests:/app tests test_service2 || exit 1
68 ===== test session starts =====
69 platform linux -- Python 3.9.21, pytest-8.3.4, pluggy-1.5.0
70 rootdir: /app
71 plugins: flask-1.3.0
72 collected 1 item
73 Service2/test_app.py . [100%]
74 ===== 1 passed in 0.16s =====
75 $ echo "Running tests for Gateway..."
76 Running tests for Gateway...
77 $ docker run --rm -v $PWD/tests/gateway_tests:/app tests test_gateway || exit 1
78 > Gateway@1.0.0 test
79 > mocha test
80 API Gateway Tests
81 gatewayPath /src/gateway
82 Skipping app.listen
83 GET /state
84 Attempting to get state
85 ✓ should return the current state
86 PUT /state
87 req.body: INIT
88 State changed to: INIT
89 ✓ should update the state to INIT
90 req.body: PAUSED
91 State changed to: PAUSED
92 ✓ should update the state to PAUSED
93 req.body: RUNNING
94 State changed to: RUNNING
95 ✓ should update the state to RUNNING
96 req.body: SHUTDOWN
97 State changed to: SHUTDOWN
98 ✓ should update the state to SHUTDOWN
99 $ passing (182ms)
100 Cleaning up project directory and file based variables
101 Job succeeded
```

Test Driven Development in the pipeline

Status	Pipeline	Created by	Stages
✖ Failed ⌚ 00:01:58 📅 1 week ago	Sync from GitHub #2712  project  55b6ea34  trigger token		  
✔ Passed ⌚ 00:03:36 📅 1 week ago	Sync from GitHub #2709  project  35a44e58  trigger token		  
✔ Passed ⌚ 00:02:23 📅 1 week ago	Sync from GitHub #2697  project  35a44e58  trigger token		  
✖ Failed ⌚ 00:02:23 📅 1 week ago	Sync from GitHub #2696  project  bf1f1b98  trigger token		  
✖ Failed ⌚ 00:02:23 📅 1 week ago	Sync from GitHub #2693  project  bf1f1b98  trigger token		  
✔ Passed ⌚ 00:01:59 📅 1 week ago	Sync from GitHub #2692  project  8158fe85  trigger token		  

Deployment in the pipeline

Status	Pipeline	Created by	Stages	
✔ Passed ⌚ 00:09:09 📅 2 hours ago	Sync from GitHub #3821  project  7d96353b  trigger token		   	
✔ Passed ⌚ 00:09:40 📅 16 hours ago	Sync from GitHub #3738  project  184ca193  trigger token		   	
✔ Passed ⌚ 00:09:45 📅 20 hours ago	Sync from GitHub #3667  project  fe3e91fa  trigger token		   	
✔ Passed ⌚ 00:08:39 📅 1 day ago	Sync from GitHub #3617  project  fe3e91fa  trigger token		   	

Broken pipeline

Status	Pipeline	Created by	Stages	
 Failed 00:05:59 2 days ago	Update .gitlab-ci.yml file #3414  project  trigger token		  	  
 Failed 00:05:08 2 days ago	Update .gitlab-ci.yml file #3413  project  trigger token		  	  
 Failed 00:05:45 2 days ago	Update .gitlab-ci.yml file #3412  project  trigger token		  	  
 Failed 00:04:24 2 days ago	Update .gitlab-ci.yml file #3411  project  trigger token		  	  
 Failed 00:04:53 2 days ago	Update .gitlab-ci.yml file #3410  project  trigger token		  	  
 Failed 00:04:30 2 days ago	Update .gitlab-ci.yml file #3409  project  trigger token		  	  
 Failed 00:04:36 2 days ago	Update .gitlab-ci.yml file #3408  project  trigger token		  	  
 Failed 00:00:50 2 days ago	Update .gitlab-ci.yml file #3406  project  trigger token		  	  
 Failed 00:01:16 2 days ago	Update .gitlab-ci.yml file #3403  project  trigger token		  	  
 Canceled 2 days ago	Update .gitlab-ci.yml file #3401  project  trigger token		  	  
 Canceled 3 days ago	Update .gitlab-ci.yml file #3400  project  trigger token		  	  
 Failed	Update .gitlab-ci.yml file --		  	  

Reflections

I learned a lot about hands-on-work with CI/CD pipelines. In this exercise simpler solution would have been a lot better where all endpoints could have been in the nginx and then that would have been extended to another service. Perhaps my solution could have been easier to extend and deploy with data persistence. I had a lot of difficulties getting the pipeline run environment to work similar to local environment, it was frustrating that the same tests pass locally and fail in the pipeline even when the file structure looks to be the same.

Synchronizing the repositories was a double edge sword. On the negative side I ran out of requests to docker hub at one point, it made the pipeline slightly more complex and wasted some computing resources as all of the stages were not mandatory. However it also helped by enforcing fixing the tests before it becomes too complex and I didn't have to worry about why some things are different in each repository.

I found it difficult to manage the pipeline. Being unable to access the docker container inside the pipeline with a terminal made me use "echoes" as way to navigate to the solution in the darkness. It

only took an ounce of my sanity to reach there. However if the pipeline wasn't running constantly maybe it would have been even more difficult to figure out later on.

Last but not least the most difficult thing was to create the deployment to cdc cloud platform. Perhaps I tried to do it in some overly complicated way and there was an easy solution, but I didn't find it. Most troubling was how to manage the differences in the configuration and then have those react to things I change somewhere else. If there is a good tutorial on how to do it, please send it to me by [email](#).

Ammount of effort

Developing the changes and understanding requirements	10
Developing the tests	8
Creating the pipeline	5
Fixing the pipeline	20 x 2
Waiting for pipeline to finish 137 times mean value of 2,5 minutes	~6
Developing deployment to cdc	40
Sum	109 hours